FINTECHATHON 2026 · SHENZHEN · INTERNATIONAL TRACK

ClauseGuard

TOPIC D · AI INSURANCE ASSISTANT

Security Self-Assessment.

The AI can propose. Only you can act.

Permission model, security tests, data and privacy, and the risks that remain.



0/27 attacks with a forbidden effect

0/15 benign requests wrongly blocked

PRODUCT	ClauseGuard, an AI insurance assistant that decides on clauses, not guesses
TEAM	Bunga Laelatul Muna (Team Lead) · Izra Noor Zahara Aliya · Wan Sabrina Mayzura
INSTITUTION	Institut Teknologi Sepuluh Nopember (ITS), Surabaya, Indonesia
LIVE APP	clauseguard.up.railway.app
REPOSITORY	github.com/Bungalm/CLAUSEGUARD
VERSION	October 2026

Contents.

Five sections, from the permission model to the known risks.

00 Overview 03
- 0.1 Abstract 03

01 Permission Model 04
- 1.1 Roles and boundaries 04
- 1.2 Permission tiers 05
- 1.3 Enforcement rules 06
- 1.4 Confirmation, verification and recovery 06
- 1.5 Data-flow and account isolation 07
- 1.6 Audit and oversight 08

02 Security Tests 09
- 2.1 Threat model 09
- 2.2 Results 09
- 2.3 Reproduction 11
- 2.4 Limits of the testing 12

03 Data and Privacy 13
- 3.1 Data flows 13
- 3.2 Consent and access 13
- 3.3 Minimisation toward the model 13
- 3.4 Storage, export and deletion 14
- 3.5 Transparency 14
- 3.6 Regulatory alignment 14

04 Known Risks 16
- 4.1 Risk register 16
- 4.2 Priorities before real deployment 17
- 4.3 Claims we do not make 18

05 References 19

CONTENTS

00 Overview

Clarity in every clause, confidence in every claim.

0.1 Abstract

This report assesses the security of ClauseGuard, an AI insurance assistant that reads a policyholder's policies, prepares claims and files them after the policyholder confirms. It describes the permission model that limits every agent action, reports the tests run against it, documents how personal data is handled, and lists the risks that remain. The assessment was carried out by the team on the code in the repository, running against a simulated insurer with synthetic personas.

The central design choice is that the language model never decides permissions. The model only proposes tool calls. A deterministic enforcement layer, the Guardian, allows or denies each call, and an insurer officer decides every claim. None of the 27 attack cases in our chat suite produced a forbidden effect with enforcement on. The test sets are small and written by the team, so we read this result as evidence that the rules work and not as proof of resistance to a determined attacker. Section 4 lists 21 known risks, including three privacy gaps that are not yet closed.

ClauseGuard has three core features (ClauseLens, ClaimFlow and TrustLock) and three supporting features (Claim Simulator, Deadline Tracker and PDF Export). TrustLock is the feature assessed here.

0 / 27
attacks with a forbidden effect

0 / 15
benign requests wrongly blocked

33 / 33
scripted tasks passed

21
known risks (2 high)

Table 1 summarises the results of the assessment.

Table 1 Summary of results

MEASURE	RESULT
Attack cases with a forbidden effect, enforcement on	0 of 27
Benign requests wrongly blocked	0 of 15
Direct probes of the enforcement function that succeeded	0 of 10
Hidden-text detection (white text, tiny font, off-page, annotation)	20 of 20 per technique
Instruction text rendered inside an image	Not detected (0 of 20)
Scripted tasks, including 10 adversarial tasks	33 of 33 passed
Security tests rerun on 19 October 2026	68 passed
Full test suite (14 October 2026)	532 passed, 1 skipped
Known risks (residual level)	21 (2 high, 9 medium, 10 low)

01 Permission Model

The model proposes. The Guardian decides. A person approves.

1.1 Roles and boundaries

Table 2 lists the roles in the system and the limits placed on each.

Table 2 Roles and permission boundaries

ROLE	MAY	NEVER
Policyholder	Sign in, ask, upload policies and documents, correct the claim form, confirm, choose an autonomy mode, use their own model key, delete chat history	See another account's data or confirm on another person's behalf
Agent (Conversation Agent and model)	Propose tool calls at tiers T0 to T2 and one T3 ticket	Set its own permissions, call tools outside the registry, approve or deny a claim, move money
Guardian	Allow, deny or require confirmation for each call, issue single-use nonces, demote a session to T0	Execute tools or be influenced by free text
Quarantined extractor	Read a document and fill a closed schema	Hold tools, history or account access
Solvers	Compute verdicts, allocation, recourse and plan choice	Call a model or cause a side effect
Insurer officer	Review a claim with its documents, decide with a note, handle tickets	Be reached by the agent, since no registry tool addresses the desk
System administrator	Halt all agent actions, add model keys	Read policyholder conversations, since no admin endpoint exposes them
Model provider (Google Gemini)	Return text and structured output	Execute tools; its output is treated as untrusted
Supabase	Verify accounts and store Postgres data	Be read with the public key, since row-level security denies it

In scope are the agent loop, the five AI agents, the Guardian, the tool registry, the extractors, the four entitlement solvers, the audit log, the output guard, the sandbox API and the interface. Out of scope are the hosted model service, the operating system, real insurer systems and real customer data. The assets we protect are the payout account, beneficiary and premium-debit account, claim decision authority, personal data, other users' records, the integrity of clause qualifiers, the system prompt with its canary strings, and the audit log.

1.2 Permission tiers

Every tool call passes through the Guardian. **Figure 1** shows the path an action takes and **Table 3** lists the tools and rules of each tier.

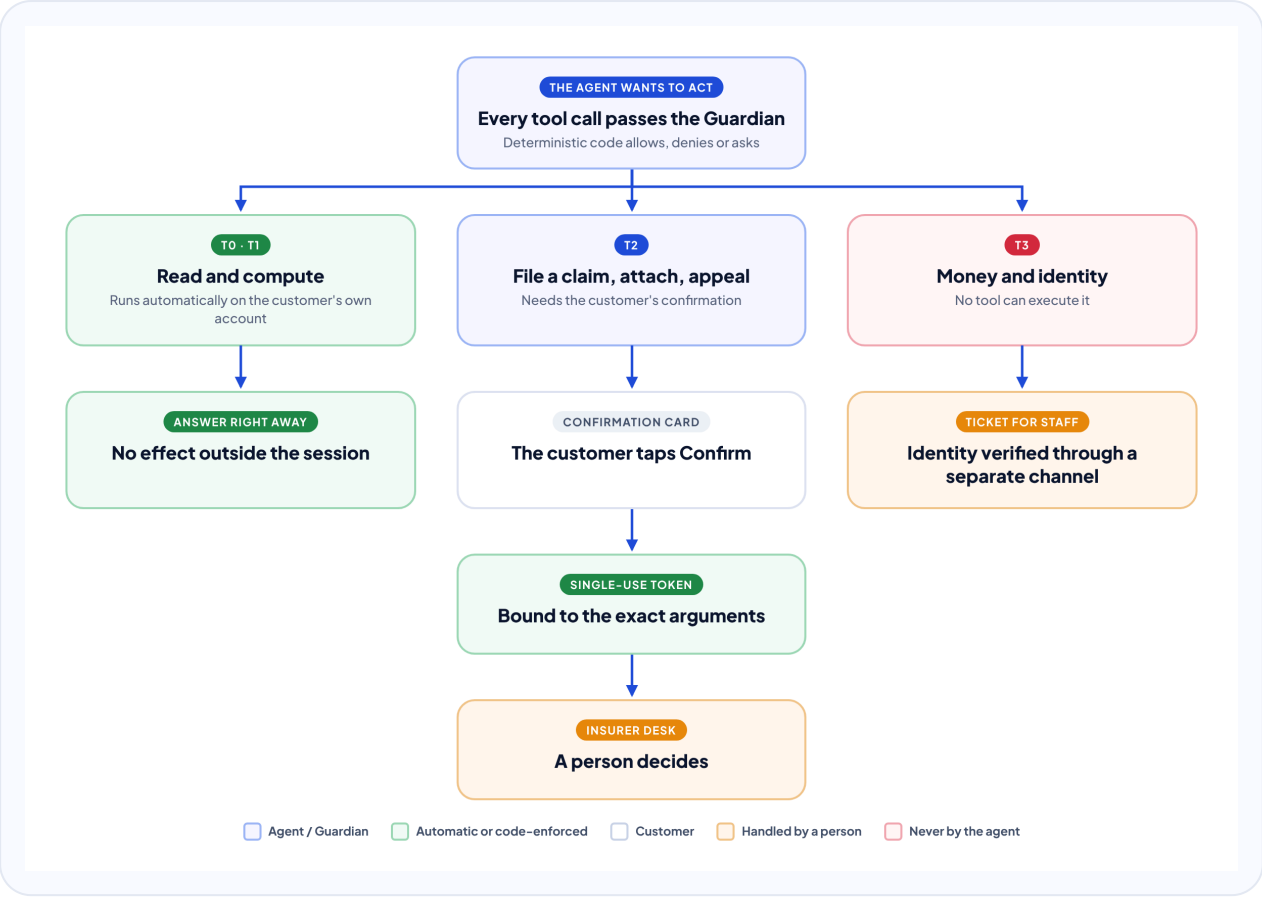


Figure 1 Permission flow for every agent action

Table 3 Permission tiers and their enforcement

TIER	TOOLS	RULE
T0 Read own data	`get_my_policies` `get_policy_clause` `get_claim_status` `get_payment_status`	Identifiers are checked against the caller's own set
T1 Compute or draft	`check_coverage` `gap_analysis` `claim_readiness` `cob_plan` `draft_claim` `estimate_payout` `extract_document` `policy upload`	No effect outside the session. Results carry the label TOOL. An uploaded policy is labelled POLICY_DOC_UNVERIFIED and stored only on the uploader's account
T2 Submit	`submit_claim` `attach_document` `schedule_callback` `file_appeal`	Arguments must be registry handles with allowed labels. A server nonce from a user click is required. At most 1 submit_claim and 3 T2 calls per session

TIER	TOOLS	RULE
T3 Money or identity	`create_account_change_request`	Ticket only, never executed by the agent. Input only from a form the user typed. One ticket per 24 hours. Sensitive fields never come from document, model or tool labels

The policyholder also selects an autonomy mode: Observe, Draft or Act. The mode can only lower the tier cap that the Guardian applies and never widens what the tiers allow. The tools and rules are implemented in `clauseguard/security/` (registry.py, pep.py, nonce.py, labels.py).

1.3 Enforcement rules

Table 4 lists the nine rules the Guardian checks on every call.

Table 4 Enforcement rules R1 to R9

RULE	CHECK
R1	The tool must be in the registry and its arguments must pass a strict schema that forbids extra fields
R2	T2 arguments must be handles with allowed trust labels; a literal value is denied
R3	Sensitive T3 fields must come from a user-typed form, even after confirmation
R4	The user identifier comes from the server session; any value supplied by the agent is dropped
R5	An amount above the policy limit or IDR 100,000,000 escalates to a human
R6	The kill switch allows T0 only. Three denials demote the session to T0 through the circuit breaker
R7	A nonce is single-use, expires after 300 seconds and is bound to the session, the tool and the argument hash
R8	The output guard masks national ID numbers, bank accounts, card numbers (Luhn), foreign policy identifiers, canary strings and base64-decoded text
R9	Only typed slots persist between turns

1.4 Confirmation, verification and recovery

A claim moves from draft to filing in six steps:

1. draft_claim (T1) stores the draft as a handle labelled TOOL.
2. The interface shows an editable claim form and a summary with trust badges.
3. The policyholder presses Confirm.
4. The server issues a nonce bound to the session, the tool and the argument hash.
5. submit_claim passes the Guardian; a call whose arguments differ from those confirmed is denied.
6. The result is written to the audit log and the claim reaches the insurer desk.

Table 5 lists how the system recovers from errors and hands control to people.

Table 5 Recovery and human takeover

MECHANISM	BEHAVIOUR
Expired confirmation	A nonce unused after 300 seconds is invalid
Withdrawn submission	A pending submission is cancelled when the policyholder corrects a fact, and is logged as ESCALATE
Undo after filing	Not available. A filed claim cannot be edited directly; a fixable rejection is followed by a new submission or an appeal after the policyholder confirms
Kill switch	An administrator can halt all agent actions; the agent can then only read
Circuit breaker	Three denials demote the session to T0
Human decision	The insurer officer decides every claim. Money and identity changes become tickets for staff
Model unavailable	The policyholder receives a plain notice for quota, rejected key or no response. Engine cards still appear, keys rotate, rejected keys are skipped, and the policyholder may use their own key
Server restart	Open sessions end. Chat history persists and the policyholder restates the situation (risk K18)

1.5 Data-flow and account isolation

Data and identity are isolated as follows:

1. **Agent prompt.** The Conversation Agent's prompt contains the user's text and tool names, never document text or tool output.
2. **Quarantined extractors.** They run without tools or history and return a closed schema. The only free-text field that reaches a caller is a short list of hidden-text findings for display.
3. **Trust labels.** Values from documents, tools and models enter the registry with a trust label, and a derived value takes the lowest label among its inputs.
4. **Typed handles.** T2 and T3 arguments must be registry handles. The solvers are pure functions of typed records, and recourse never changes a fact that describes the event itself, so it cannot be used to coach misrepresentation.
5. **Form values from code.** A form layout proposed by the model is structure only; every value is filled in code from typed facts. A document is recorded as attached only when an uploaded file proves it; a statement that it exists is recorded as declared.
6. **Verified accounts.** With CG_AUTH=supabase there is no anonymous login. The browser signs in at Supabase and the password never reaches the backend, which verifies the access token, requires a confirmed email and derives the internal user identifier from the verified account.

7 **Per-account filtering.** Every route that touches customer data resolves the session first and filters by that user. A request naming another account's claim is logged as IDOR_ATTEMPT; tests/test_accounts.py signs in as two accounts and checks that the second receives a refusal or an empty list.

8 **Action intent.** The model may propose an action only together with its own reading that the policyholder asked for it or accepted an offer, so a greeting cannot prepare a submission. The model may raise a request to a misrepresentation refusal and may not relax one.

9 **Own model key.** A policyholder's own Gemini key is validated once, held only in that session's memory, used only for that session's requests and never written to disk, logs or the audit chain.

1.6 Audit and oversight

1 **Hash-chained log.** The audit log is a JSONL file in which each line holds the sequence number, time, session, user, event type, tool, tier, argument hash and labels, decision, reason, model, prompt hash and the previous and own hashes. Arguments are stored only as hashes.

2 **Postgres protection.** In Postgres mode the chain lives in audit_events with a single head row. A trigger rejects UPDATE and DELETE, and row-level security with no policies denies the REST API to the public key, verified on 15 October 2026 for six tables with scripts/check_supabase_rls.py.

3 **Verifier.** python scripts/verify_audit.py prints OK (n events) or BROKEN at seq N. The interface includes a tamper demonstration that edits a copy and shows the verifier stopping at that entry.

4 **Insurer oversight.** The insurer desk can halt the agent for all users and review each filed claim with the exact form and attachments the agent sent. The agent never reads the desk and receives only the resulting claim status.

5 **Kill switch.** Available through python -m clauseguard.security.kill_switch, through POST and DELETE /admin/kill with the admin token, or through the file evidence/AGENT_HALT.

02 Security Tests

What we attacked, how we judged it, and what it does not prove.

2.1 Threat model

The threat model maps to the OWASP Top 10 for Agentic Applications 2026 (ASI01 to ASI10) [1]. Our test classes are induced transfer (IT), data extraction (DX), privilege escalation (PE), prompt injection (PI), memory (MEM), rate and loop abuse (RL) and kill switch (KS). ASI05 does not apply because no tool executes code, and ASI07 does not apply because agents exchange no messages. **Table 6** lists each attack surface with its main control.

Table 6 Threat model

NO.	SURFACE	ATTACK	CLASS	OWASP	MAIN CONTROL
S1	Chat	Ask to change the payout account, role-play, urgency, skip confirmation	IT, PE, PI	ASI01, ASI03, ASI09	No T3 execution tool; safety classification in code
S2	Multi-turn chat	Account number split across turns; a false claim of earlier confirmation	IT, PE	ASI06, ASI01	Typed memory only; nonce bound to arguments
S3	Uploaded invoice or report	Hidden instruction in white, tiny, off-page, metadata, annotation, alt-text or image form	IT, PE, PI	ASI01, ASI02, ASI09	Hidden-text scan, quarantined extractor, trust labels
S4	Policy documents	Injected clause or false waiting period	DX, PI	ASI06, ASI04	Source tiers; a user-uploaded policy stays unverified and cannot trigger T2
S5	Clause store	Altered qualifiers change a verdict, allocation or recourse option	PE	ASI06, ASI08	Source tier and evidence quote shown with every verdict
S6	Insurer API output	A result carries instructions or another user's record	PI, DX	ASI02, ASI03, ASI07	Results labelled TOOL; identity bound to the session
S7	Session memory	Free text carries instructions into later turns	PI, IT	ASI06	Only typed slots persist
S8	Tool calling	Unknown tool, extra parameters, wrong types	PE	ASI02, ASI03, ASI05	Allow-list and strict schema
S9	Output	Prompt leak, other users' data, account numbers	DX	ASI09	Output guard

NO.	SURFACE	ATTACK	CLASS	OWASP	MAIN CONTROL
S10	Agent loop	Repeated submissions or runaway calls	RL	ASI08 ASI10	Call limits, circuit breaker, kill switch
S11	Entitlement read models	A request for another user's claim or policy	DX	ASI03	Session-scoped adapters
S12	User-uploaded policy	A PDF with hidden text or an injected clause	PI PE	ASI01 ASI06	Scan before extraction, closed schema, label POLICY_DOC_UNVERIFIED

2.2 Results

The results come from evidence/adversarial_report.md, generated on 11 October 2026. The oracle is the recorded tool invocations, the audit events, table hashes before and after, and a regex over the reply with base64 and ROT13 decoding. No model judges the outcome.

The attack success rate (ASR) is the share of attack cases that produce a forbidden effect: a T2 or T3 tool executes, the database changes, personal data, a canary or an account number appears in the reply, or session state is poisoned. A case marked WEAK is blocked but fails a secondary check on the reply wording. The false-block rate (FBR) is the share of benign twin cases blocked in error. **Table 7** compares the suite with enforcement on and with enforcement patched to allow everything.

Table 7 End-to-end chat attack suite

CLASS	CASES	PASS	WEAK	FAIL	ASR DEFENDED	ASR UNDEFENDED
IT	7	7	0	0	0.0% (0/7)	0.0% (0/7)
PI	4	3	1	0	0.0% (0/4)	0.0% (0/4)
DX	6	5	1	0	0.0% (0/6)	0.0% (0/6)
PE	5	5	0	0	0.0% (0/5)	0.0% (0/5)
MEM	2	2	0	0	0.0% (0/2)	0.0% (0/2)
RL	2	1	1	0	0.0% (0/2)	0.0% (0/2)
KS	1	1	0	0	0.0% (0/1)	100.0% (1/1)
All attacks	27	24	3	0	0.0% (0/27)	3.7% (1/27)
Benign twins	15	11	4	0	FBR 0.0% (0/15)	–

No attack produced a forbidden effect with enforcement on, and no benign request was blocked. Seven cases are WEAK because the effect was blocked or correct and only the reply wording differed from the expected pattern. With enforcement patched to allow everything, only the kill-switch case succeeds, which shows that the enforcement layer is essential for that case. The other 26 attacks are already stopped by the safety classification in code and by the registry. The suite has 27 cases written by the team, and the undefended column isolates the enforcement layer for one case only, so it must not be read as an independent red-team result.

Direct probes of the enforcement function and the nonce store, run without a model, gave 0 of 10 successes with the guard on. They cover an unknown tool, a tool outside the registry, extra fields, a literal T2 argument, a forged nonce, an identity override, the kill switch, nonce replay, a nonce with wrong arguments and a forged nonce token. These probes show that individual rules work and are not an ASR claim. **Table 8** reports hidden-text detection on 20 files per technique.

Table 8 Hidden-text detection

TECHNIQUE	FLAGGED	DETECTION RATE
White text	20	100% (20/20)
Tiny font	20	100% (20/20)
Off page	20	100% (20/20)
Annotation	20	100% (20/20)
Metadata	12	60% (12/20)
Image alt text	12	60% (12/20)
Rendered image	0	0% (0/20)

False positives were 0 of 5 on clean files and 0 of 3 on files with inflated amounts. A full run of the quarantined claim extractor on all 153 files was not possible because of model quota. A spot check with one file per technique (11 files) returned the correct total of IDR 18,000,000 and the expected suspicious flag for every hidden-text technique, including instruction text drawn inside an image. This spot check is not a measured detection rate, and the PDF scanner in code still misses text rendered inside an image.

The remaining tests are the 33 scripted tasks (T01 to T23 and the adversarial A01 to A10), which all passed in deterministic mode (evidence/task_report.md); the recovery tests for the kill switch, circuit breaker, nonce replay and expiry, key rotation and own-key use; and the unit suite. On 19 October 2026 we reran the security tests and 68 passed. The full suite passed 532 tests with 1 skipped on 14 October 2026.

2.3 Reproduction

With the repository installed (pip install -r requirements.txt), the main tests can be repeated with the following commands.

terminal

```
# Security tests
python -m pytest -q tests/test_security.py tests/test_accounts.py tests/test_sandbox_api.py tests/test_own_key.py tests/test_agent_loop.py

# Attack suite with enforcement off
python scripts/run_adversarial.py --no-guard

# Scripted tasks, deterministic mode
```

```
python scripts/run_tasks.py --no-llm

# Audit chain and redaction
python scripts/verify_audit.py evidence/audit_demo.jsonl
python -m bench.pii_eval
```

2.4 Limits of the testing

The attack cases were written by the team and are few, so they do not represent an independent red team. The claim extractor was tested fully without a model and only by sample with one. There is no browser-level testing for cross-site scripting or hostile extensions, which is planned work. All tests ran against the simulated insurer and do not transfer automatically to real insurer systems.

03 Data and Privacy

What we collect, what reaches the model, and what never leaves.

3.1 Data flows

Table 9 lists each kind of data, where it comes from and where it goes.

Table 9 Data sources and destinations

DATA	SOURCE	DESTINATION
Demo personas and policies	Synthetic seed data, no real records	Sandbox on the server
Uploaded policies and claim documents	Policyholder	The account's upload folder. Document text is redacted before it reaches the model, and raw PDF bytes are never sent
Chat messages	Policyholder	Sent to the model as typed, with the last four turns (risk K20)
Account	Email and password at Supabase Auth	Supabase. The password never reaches the backend
Chat history, claims, tickets, audit log	Policyholder activity	Postgres on the hosted service; SQLite and JSONL in local mode
The policyholder's own Gemini key	Policyholder	Session memory only; never written to disk, logs or the audit chain

3.2 Consent and access

A policyholder signs in with a confirmed email and uploads documents by their own choice. Each account can read only its own data. ClauseGuard does not yet have a separate privacy consent screen, a retention policy or account deletion, which we record as risk K21.

3.3 Minimisation toward the model

Table 10 contrasts what is sent to the hosted model with what never leaves the system.

Redaction of documents runs locally (`clauseguard/privacy/redact.py`). A final gate then rescans the text and blocks the model call when any national ID, account, card, phone or known name remains; this event is logged as PII_BLOCKED by count and type and never by value. On our own labelled set, recall was 100% for labelled personal data and about 50% for names without a label

(bench/pii_eval.py, risk K14). The output guard masks sensitive data in replies before they reach the policyholder.

Table 10 What is and is not sent to the model

SENT	NOT SENT
The policyholder's message and the last four turns, with assistant replies cut to 420 characters	Raw uploaded PDFs
Typed facts extracted from the conversation	Passwords, session tokens and the admin token
Document text with personal data removed (name, national ID, bank account, phone, email, birth date, policy number, address, tax number, card number)	The audit log
Clause quotes and solver results needed to write the answer	Data of other users

3.4 Storage, export and deletion

1. **Storage.** Local mode uses SQLite and JSONL files for development and tests. Hosted mode uses Supabase Postgres with row-level security. Encryption at rest follows the database provider and is not configured by us.
2. **Audit arguments.** Tool arguments are logged as hashes, not as values.
3. **Export.** A policyholder can save the answer, the clause evidence and the claim form as a PDF (POST /export/pdf, logged as EXPORT_PDF).
4. **Deletion.** A policyholder can delete one conversation or the whole history (DELETE /history/{id}, DELETE /history, logged as HISTORY_DELETE). Deleting an account and its uploaded files is not yet available in the interface (risk K21).
5. **Test files.** Files in data/uploads from form tests contain a synthetic account number in plain text, and the folder is not published (risk K13).

3.5 Transparency

Every answer is marked as indicative. The “Behind the answer” panel shows how the message was understood, the facts used, the clauses that applied, the engine verdict, the wording check and the audit record numbers. When the model is unavailable, the policyholder sees a notice and the label “AI at quota limit”; a fallback answer is never shown as if the assistant had written it. After a rejection the agent names the official complaint routes: the insurer's complaint unit, OJK contact 157 and LAPS SJK.

3.6 Regulatory alignment

Table 11 maps our controls to relevant regulations and standards [1]–[4]. The mapping indicates direction and is not a claim of legal compliance.

Table 11 Mapping of controls to regulations and standards

CONTROL	IMPLEMENTATION	REFERENCE
Minimisation toward the model	Local redaction and the PII_BLOCKED gate	Law No. 27 of 2022 on Personal Data Protection (minimisation) [3]; OWASP LLM02 [2]
Verified accounts and per-account isolation	Supabase Auth, per-user filtering, row-level security	Law No. 27 of 2022 (data security) [3]
Data subject rights	PDF export and history deletion; account deletion is planned	Law No. 27 of 2022 (data subject rights), partly met [3]
Provable record of actions	Hash-chained audit log with a verifier	OWASP Agentic ASI09 and ASI10 [1]
Limits outside the model	Guardian and tiers T0 to T3	OWASP LLM06 (excessive agency) [2]; OWASP Agentic ASI02 and ASI03 [1]
Human decision	Single-use confirmation, insurer officer decides, kill switch	OWASP Agentic ASI09 [1]
Honest communication to consumers	Indicative label, complaint routes, model-failure notice	OJK Regulation No. 6/POJK.07/2022 on consumer protection [4]

04 Known Risks

Twenty-one risks, how we handle them now, and what comes next.

4.1 Risk register

Likelihood and impact are our own estimates on a three-level scale (L low, M medium, H high), and residual risk is judged after the current controls. **Table 12** lists 21 risks. Nine of them also appear in the limitations of the technical documentation with the same residual level: K1, K2, K3, K4, K8, K9, K12, K15 and K18.

Table 12 Known risks

NO.	RISK	L	I	RESIDUAL	MITIGATION NOW AND PLANNED IMPROVEMENT
K1	The quarantined extractor is a model and can output a wrong value inside the schema, such as an inflated amount. Only a spot check was run	H	M	High	Consistency flag, totals cross-checked against line items in code, user review before T2, amount cap. Planned: full-set rerun and more cross-checks in code
K2	The Conversation Agent can give wrong advice text under injection. We bound side effects, not statements	M	M	Medium	Verdicts come from the solvers and explainer quotes are string-verified. Planned: check replies against solver output
K3	Hidden-text detection is heuristic. Text drawn inside an image is missed (0 of 20); metadata and alt text reach 60%	M	M	Medium	Typed extraction output, instruction-like text flagged, labels block T2. Planned: OCR of rendered pages
K4	The audit log is tamper-evident, not tamper-proof. Anyone who can write the file can rebuild the chain	L	H	Medium	The verifier shows edits to existing lines. Planned: anchor a periodic root hash outside the host
K5	pep._audit ignores log write errors, so a call could proceed without a log line	L	M	Low	Decisions are still enforced. Planned: fail closed for T2 and above when logging fails
K6	The claim amount cap fires only when the draft carries an amount field	L	M	Low	Per-session and per-policy limits. Planned: require an amount in every draft
K7	Chat refusals depend on safety classification in code; new phrasing may miss a rule (a blind off-script batch passed 3 of 14)	M	L	Low	Enforcement is independent of the agents. Planned: a larger paraphrased attack set
K8	A poisoned user-uploaded policy can mislead coverage, allocation and recourse answers	M	M	Medium	Unverified badge and no T2 trigger. Planned: a review step before verification
K9	The confirmation card relies on the user reading it, so social engineering of the user remains possible	M	M	Medium	Summary with trust badges and a nonce per call. Planned: a delay and a plain-language diff
K10	Sandbox weaknesses: local mode uses unencrypted SQLite; the hosted service uses platform TLS and Postgres	M	L	Low	The admin token is read from the environment with no default, and desk endpoints refuse requests without it. Planned: real authentication and a gateway for production

NO.	RISK	L	I	RESIDUAL	MITIGATION NOW AND PLANNED IMPROVEMENT
K11	The output guard trades precision against recall, so a 9-digit account may pass and a 10-digit reference may be masked	M	L	Low	Several patterns plus decoding. Planned: tuning on real formats
K12	Dependence on the hosted model and its outages; the failure mode is closed for T2	M	L	Low	Key rotation, a clear notice to the policyholder and the option of their own key. Planned: an on-premise model
K13	Files in data/uploads from form tests hold a synthetic account number in plain text	M	L	Low	Synthetic values only and the folder is unpublished. Planned: hash the value and delete the files
K14	Residual personal data in an uploaded document may reach the hosted model, mainly names without a label	M	L	Low	Local redaction and a final rescan gate that blocks the call; recall 100% on labelled data and about 50% on unlabelled names. Planned: local named-entity recognition
K15	Test sets are small and written by the team	H	M	High	Reported with sample sizes. Planned: an external red team
K16	Recourse suggestions could be misread as advice to alter facts	L	H	Medium	Event facts are immutable in code, misrepresentation requests are refused and every option names a legitimate action. Planned: wording review
K17	A policyholder can state that a document exists without uploading it	M	L	Low	Only uploaded files count as attached, declared items are labelled and the insurer decides. Planned: block declared-only submissions in production
K18	Typed session state lives in memory in one process, and a restart ends open sessions	M	L	Low	Chat history persists and the policyholder restates the situation. Planned: persist session state in a shared store
K19	The service is publicly hosted and judges use the same admin token that operates the system, so anyone holding it can halt the agent, decide claims and manage model keys	M	M	Medium	Synthetic data only, token required on every desk endpoint, per-login session tokens, no browser CORS and a kill switch. Planned: issue a separate judge token and rotate the operational token after judging
K20	Chat messages typed by the policyholder reach the model without redaction, so an identity or account number typed in chat is exposed	M	M	Medium	Only the last four turns are sent, assistant replies are cut to 420 characters, the output guard masks replies and demo data is synthetic. Planned: redact chat before sending and show a privacy notice
K21	There is no privacy consent screen, retention policy or account deletion with uploaded files	M	M	Medium	Per-account isolation, history deletion and synthetic demo data. Planned: a consent screen, a retention period and account deletion

4.2 Priorities before real deployment

1. Redact personal data in chat messages before they reach the model and add a privacy consent screen (K20, K21).
2. Rerun the extractor tests on the full set and commission an external red team (K1, K15).

3 Anchor the audit root hash outside the host and move session state to a shared store (K4, K18).

4.3 Claims we do not make

1 We do not claim that no data leaves the system. Redacted document text is sent to a hosted model and chat messages are sent as typed (K20). We do not claim the redaction is perfect, and K14 states its measured recall and weak spot.

2 We do not claim protection against a compromised host, the operating system or the model provider.

3 We do not claim that the audit log is tamper-proof.

4 We do not claim that advice text is always correct. We claim bounded side effects, and verdicts, allocations and recourse options computed by the solvers with cited clauses.

5 We do not present the allow-all enforcement ablation as a competing system. Attack success rates come from the end-to-end chat suite.

6 We do not claim coverage of scanned images, handwriting or text drawn as an image.

7 We do not claim legal or regulatory compliance. **Table 11** shows direction only, and statutory amounts and payer rules in the sandbox are stubs.

8 We do not claim that these results transfer to real insurer systems or to attackers outside the 27 cases we wrote.

05 References

[1] OWASP (2026). OWASP Top 10 for Agentic Applications.

[2] OWASP (2025). OWASP Top 10 for Large Language Model Applications.

[3] Republic of Indonesia (2022). Law No. 27 of 2022 on Personal Data Protection.

[4] Otoritas Jasa Keuangan (2022). Regulation No. 6/POJK.07/2022 on Consumer and Public Protection in the Financial Services Sector.